

Instituto Superior de Engenharia de Lisboa

Mestrado em Engenharia Informática e Multimédia

Computer Vision and Mixed Reality

Marker Based Augmented Reality

Bruno Rodrigues — 52323

Docente: Pedro Mendes Jorge

Junho 2026

Índice

Introdução	3
Dependências	3
Como correr	3
1. Calibrar a câmara	3
2. Testar deteção ArUco (opcional)	3
3. Aplicação AR principal	4
Passos manuais pendentes (fazer antes de correr pela primeira vez)	4
A. Imprimir o tabuleiro de xadrez para calibração	4
B. Gerar e imprimir os marcadores ArUco	4
C. Tirar as fotos de calibração	4
Como funciona o código	5
Fluxo geral	5
calibrate.py	5
aruco_detector.py	5
main.py	6
Estrutura do projeto	6
Problemas encontrados e lições aprendidas	7
Deteção ArUco — borda branca obrigatória	7
Calibração — a rigidez do tabuleiro é crítica	7
Conclusão	8

Introdução

O presente relatório descreve o desenvolvimento do segundo projeto da unidade curricular de Computer Vision and Mixed Reality (VARM), do Mestrado em Engenharia Informática e Multimédia do ISEL.

O objetivo do projeto é a implementação de uma aplicação de Realidade Aumentada (AR) baseada em marcadores ArUco, utilizando a biblioteca OpenCV em Python. A aplicação permite detetar marcadores impressos em tempo real através de uma webcam e sobrepor objetos virtuais tridimensionais alinhados com cada marcador, criando a ilusão de que os objetos existem no espaço físico.

O projeto divide-se em três componentes principais. A primeira é a calibração da câmara, processo que determina os parâmetros intrínsecos da câmara (matriz K e coeficientes de distorção) a partir de imagens de um tabuleiro de xadrez, transformando a câmara num instrumento de medição capaz de relacionar distâncias reais com píxeis. A segunda componente é a deteção de marcadores ArUco e a estimação de pose, que permite determinar a posição e orientação de cada marcador no espaço tridimensional relativamente à câmara. A terceira é o registo de objetos virtuais, onde se projeta geometria 3D (cubo e pirâmide) sobre a imagem da câmara de forma a ficar alinhada com o marcador correspondente.

A integração com o motor de jogo Unity3D, prevista como componente opcional no enunciado, não foi realizada no âmbito deste projeto.

Dependências

```
pip install opencv-contrib-python==4.9.0.80 numpy==1.26.4
```

Usar opencv-contrib-python (não opencv-python) para ter acesso ao módulo cv2.aruco.

Como correr

1. Calibrar a câmara

```
python calibrate.py
```

Controlos: SPACE captura frame · c calibra e guarda · q sai

Gera camera_calibration.npz com os parâmetros intrínsecos da câmara.

Para verificar o resultado (mostra feed original vs sem distorção):

```
python calibrate.py --check
```

2. Testar deteção ArUco (opcional)

```
python aruco_detector.py
```

Mostra o feed da câmara com eixos de pose desenhados sobre cada marcador detetado.

3. Aplicação AR principal

python main.py

Sobreposição objetos 3D wireframe sobre os marcadores detetados:

ID	Objeto
0	Cubo (ciano)
1	Pirâmide (verde)
outro	Eixos XYZ

Passos manuais pendentes (fazer antes de correr pela primeira vez)

O código está completo mas é necessário preparar o material físico antes de executar.

A. Imprimir o tabuleiro de xadrez para calibração

1. Usar um tabuleiro com **9×6 cantos internos** (10×7 quadrados)
2. Imprimir em A4 e colar numa superfície rígida plana (cartão, pasta)
3. Medir o tamanho real de um quadrado em metros e atualizar SQUARE_SIZE em calibrate.py
 1. Default: 0.025 (2.5 cm) — ajustar conforme a impressão
4. Tabuleiro de exemplo: <https://docs.opencv.org/master/pattern.png>

B. Gerar e imprimir os marcadores ArUco

python generate_markers.py

Gera markers/marker_0.png e markers/marker_1.png (dicionário 6×6_250).

1. Tamanho sugerido: 5–8 cm de lado
2. Após imprimir, medir o lado real e atualizar MARKER_SIZE em aruco_detector.py e main.py
 1. Default: 6.0 (6 cm) — ajustar conforme a impressão

C. Tirar as fotos de calibração

1. Com o tabuleiro impresso e a webcam ligada, correr python calibrate.py
 2. Capturar **mínimo 10 frames** com o tabuleiro em posições, ângulos e distâncias variados
 3. O erro de reprojeção (RMS) ideal é abaixo de **1.0 px**
-

Como funciona o código

Fluxo geral

calibrate.py

```
-- captura fotos do chessboard
-- calibrateCamera()
-- guarda camera_calibration.npz
```

main.py (cada frame):

```
webcam → gray
-- detectMarkers() → corners, ids
-- solvePnPRansac(MARKER_OBJ_PTS, corners_2D, mtx, dist) → rvec, tvec
-- Rodrigues(rvec) → R (matriz 3×3)
-- projectPoints(vertices_3D, rvec, tvec, mtx, dist) → pixels_2D
-- cv2.line() por cada aresta → objeto na imagem
```

A chave conceptual é: **a calibração transforma a câmara num instrumento de medição**. Sem ela, o solvePnPRansac não consegue calcular distâncias reais nem orientações corretas, e os objetos não ficam alinhados com o marcador.

calibrate.py

objp — grelha de 54 pontos 3D no plano $Z=0$ com coordenadas reais (em metros). São as posições conhecidas dos cantos do tabuleiro no mundo físico.

capture_mode() — cada frame converte para cinzento e chama findChessboardCorners. Se encontrar os 54 cantos, ao premir SPACE refina-os com cornerSubPix (precisão subpixel) e guarda o par: pontos 3D reais ↔ pixels 2D na imagem.

_run_calibration() — com N pares (3D→2D), calibrateCamera resolve o sistema e devolve: - **mtx** — matriz intrínseca K com f_x , f_y (distâncias focais) e c_x , c_y (centro óptico) - **dist** — 5 coeficientes de distorção radial/tangencial - **rms** — erro médio de reprojeção em píxeis (bom abaixo de 1.0 px)

check_mode() — mostra original vs undistort() lado a lado para verificar visualmente.

aruco_detector.py

```
aruco_dict = cv2.aruco.getPredefinedDictionary(cv2.aruco.DICT_6X6_250)
detector = cv2.aruco.ArucoDetector(aruco_dict, DetectorParameters())
corners, ids, _ = detector.detectMarkers(gray)
rvecs, tvecs, _ = cv2.aruco.estimatePoseSingleMarkers(corners, MARKER_SIZE, mtx, dist)
```

DICT_6X6_250 = dicionário com 250 marcadores possíveis, cada um codificado numa grelha 6×6 de bits.

Para cada marcador detetado: - **rvec** — vetor de rotação de Rodrigues (direção = eixo de rotação, magnitude = ângulo) - **tvec** — posição do centro do marcador em metros no referencial da câmara $[x, y, z]$ - $\text{np.linalg.norm}(\text{tvec})$ = distância euclidiana da câmara ao marcador

`drawFrameAxes` desenha os eixos XYZ sobre o marcador para visualizar a orientação.

main.py

Os objetos são definidos como vértices no **espaço do marcador** ($Z=0$ = plano do marcador, Z cresce para cima) e arestas entre eles:

```
# Cubo: 4 vértices base (Z=0) + 4 vértices topo (Z=h)
# Pirâmide: 4 vértices base + 1 ápice em (0, 0, h)
```

Estimação de pose por marcador — em vez de `estimatePoseSingleMarkers`, usa-se explicitamente `solvePnPRansac` + Rodrigues:

```
ok, rvec, tvec, _ = cv2.solvePnPRansac(MARKER_OBJ_PTS, corner[0], mtx, dist)
R, _ = cv2.Rodrigues(rvec) # vetor de rotação → matriz 3×3
```

`MARKER_OBJ_PTS` são os 4 cantos do marcador em 3D (espaço do marcador, cm). O RANSAC torna a estimação robusta a cantos mal detetados.

draw_object() — usa `projectPoints` para transformar os vértices 3D em píxeis 2D:

```
projected, _ = cv2.projectPoints(vertices, rvec, tvec, mtx, dist)
```

Transformação completa: espaço do marcador → espaço da câmara (via `rvec/tvec`) → plano da imagem (via `mtx`) → correção de distorção (via `dist`). Depois liga os vértices com `cv2.line`.

O objeto “segue” o marcador porque `rvec/tvec` são recalculados a cada frame.

OBJECTS dict — mapeia ID → função construtora do objeto:

```
OBJECTS = {0: _cube_edges, 1: _pyramid_edges, 2: _nova_funcao}
```

Estrutura do projeto

p2/

```
— calibrate.py      # calibração da câmara com chessboard
— aruco_detector.py # deteção ArUco + estimação de pose standalone
— main.py          # aplicação AR principal (solvePnPRansac + Rodrigues)
— generate_markers.py # gera imagens dos marcadores ArUco
— markers/         # imagens geradas (marker_0.png, marker_1.png)
— prints/          # screenshots do resultado
— camera_calibration.npz # gerado após calibrar (não commitar)
```

Problemas encontrados e lições aprendidas

Deteção ArUco — borda branca obrigatória

Os marcadores impressos não tinham margem branca à volta. O algoritmo de deteção procura a transição preto→branco na borda exterior do marcador para identificar os quadrados negros — sem essa transição, simplesmente não deteta nada (sem erro, sem aviso).

Solução: colar o marcador num papel/cartão branco com pelo menos 1–2 cm de margem em todos os lados, ou reimprimir com margem suficiente.

Calibração — a rigidez do tabuleiro é crítica

A calibração exige imagens do tabuleiro em posições, distâncias e ângulos variados. Ao longo do processo testámos três abordagens, com resultados muito diferentes:

1.ª tentativa — só rotação a 90° face à câmara Manter o tabuleiro sempre perpendicular à câmara e apenas rodar o plano não dá variedade angular suficiente. O algoritmo não consegue estimar bem os parâmetros de distorção radial porque todos os pontos ficam numa zona estreita da imagem. RMS elevado e instável.

2.ª tentativa — papel sem suporte rígido Tentar vários ângulos com o papel solto faz com que o papel curve ligeiramente. Os cantos do tabuleiro ficam num plano ligeiramente curvo em vez de plano — o `calibrateCamera` assume que $Z=0$ para todos os pontos, por isso qualquer curvatura do papel introduz erro sistemático. O RMS ficou alto (>9 px em alguns casos).

3.ª tentativa — tabuleiro colado em superfície rígida (resultado final) Colar o papel numa superfície dura (pasta, cartão espesso) garante que os cantos ficam rigorosamente no mesmo plano. Com 67 frames capturadas em ângulos, distâncias e inclinações variados, obtivemos $RMS = 2.41$ px — o melhor resultado da sessão.

Conclusão: a rigidez do suporte é tão importante quanto a variedade de ângulos. Um tabuleiro que curve mesmo ligeiramente degrada significativamente a calibração.

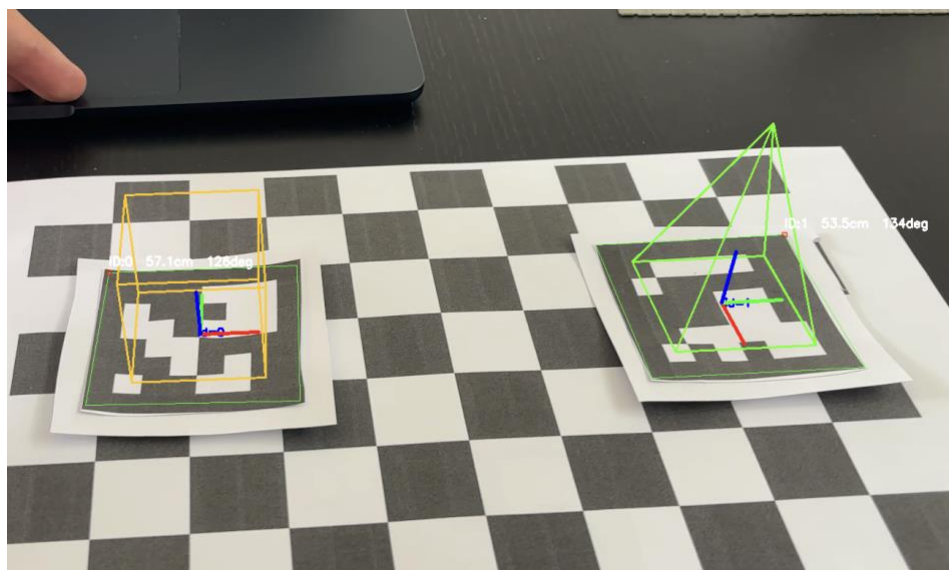
Conclusão

O projeto foi concluído com sucesso, cumprindo todos os requisitos obrigatórios definidos no enunciado. A aplicação é capaz de detetar marcadores ArUco em tempo real, estimar a sua pose com `solvePnP` e `Rodrigues`, e sobrepor objetos 3D wireframe (cubo para o ID 0, pirâmide para o ID 1) alinhados com cada marcador.

O principal desafio encontrado foi a calibração da câmara. Verificou-se que a qualidade da calibração depende fortemente da rigidez do suporte do tabuleiro de xadrez — um tabuleiro impresso em papel solto introduz curvatura no plano de referência, degradando significativamente o erro de reprojeção (RMS superior a 9 px em testes com papel solto, contra 2.41 px com tabuleiro colado em superfície rígida). A variedade de ângulos e distâncias de captura revelou-se igualmente crítica para uma boa cobertura dos parâmetros de distorção radial.

Um segundo problema identificado foi a necessidade de margem branca em redor dos marcadores impressos: sem essa margem, o algoritmo de deteção ArUco não encontra a transição preto → branco necessária para identificar os quadrados do marcador, falhando silenciosamente.

Em termos de aprendizagens, o projeto permitiu compreender na prática o pipeline completo de AR baseada em marcadores: desde a calibração que estabelece a relação entre o mundo real e a imagem, passando pela estimação de pose que posiciona o marcador no espaço 3D, até à projeção de vértices que fecha o ciclo ao mapear os objetos virtuais de volta para píxeis na imagem.



ID0 – Cubo e ID1- Piramide